Desarrollo de Aplicaciones Web

Formato Datos: XML, JSON

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Pablo García Tahoces

XML

Su nombre deriva de *eXtensible Markup Languaje*

- Como su nombre indica es un lenguaje de marcas, mantenido por W3C.

- Es un lenguaje muy útil para describir información estructurada.

- Facilitan el intercambio de información entre usuarios no humanos (máquinas)

- Existen herramientas desarrolladas por los principales fabricantes (Microsoft, Apache, …) que facilitan su uso.

XML

Archivos de Propiedades *vs* Archivos XML

Archivo de propiedades

Nombre= Valor
fontname= Times Roman
fontsize= 12
windowsize= 400 200
color= 0 50 100

Tienen jerarquía plana

Nombre= Valor
titulo.fontname= Times Roman
titulo.fontsize= 12
cuerpo.windowsize= 400 200
cuerpo.color= 0 50 100

No admiten repetición de valores

Nombre= Valor
menu.opción.1= Times Roman
menu.opción.2= Helvetica
menu.opción.3= Goudy Old Style

XML

Un archivo XML, permite expresar estructuras jerárquicas, mediante el uso de etiquetas.

```
<configuration>
        <title>
                <font>
                        <name>Helvetica</name>
                        <size>36</size>
                </font>
        </title>
        <body>
                <font>
                        <name>Times Roman</name>
                        <size>12</size>
                </font>
        </body>
        <window>
                <width>400</width>
                <height>200</height>
        </window>
        <color>
                <red>0</red>
                <green>50</green>
                <blue>100</blue>
        </color>
        <menu>
                <option>Times Roman</option>
                <option>Helvetica</option>
                <option>Goudy Old Style</option>
        </menu>
</configuration>
```

XML *vs* HTML

- XML distingue entre mayúsculas y minúsculas.
- En XML, siempre ha de incluirse el marcador final.
- En XML, los elementos con un único marcador han de acabar con “/>”.
- En XML, los valores de los atributos pueden ir encerrados entre comillas dobles o simples, pero ha de abrirse y cerrarse el mismo tipo de comillas.
- En XML, todos los atributos tienen que tener un valor.

Nota.- se denomina XHTML, el HTML que sigue unas reglas basadas en XML.

XML. Estructura de un documento.

- Encabezado (opcional, recomendado)

```
<?xml version= “1.0” encoding=“UTF-8”?>
```

- Definición de tipo de documento (opcional)

```
<!DOCTYPE web-app PUBLIC
        “-//Sun Microsystems, Inc.//DTD Web Application 2.2//EN”
        http://java.sun.com/j2ee/dtds/web-app_2_2.dtd>
```

- Cuerpo (incluye elementos hijos, texto o ambas cosas. Son los elementos básicos)

```
<configuration>
        <title>
                <font>
                        <name>Helvetica</name>
                        <size>36</size>
                </font>
        </title>
</configuration>
```

XML. Consejos.

- Se debe evitar el contenido mixto (texto y elementos, al mismo nivel).

```
<font>
        Helvetica
        <size>36</size>
</font>
```

```
<font>
        <name>Helvetica</name>
        <size>36</size>
</font>
```

- Utilizar atributos únicamente para modificar la interpretación de un valor, no para especificar valores.

```
<font name="Helvetica" size="36 pt"/t>
```

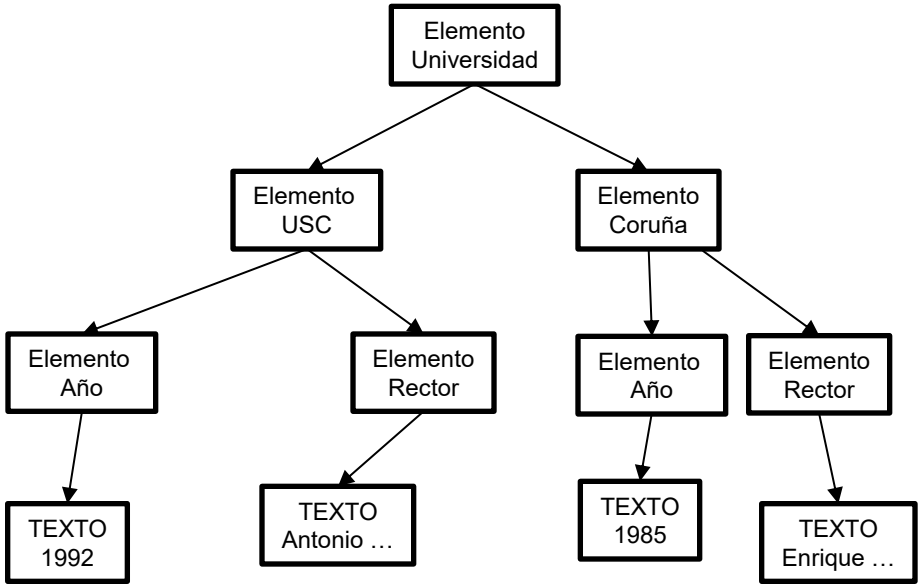
```
<font>
        <name>Helvetica</name>
        <size unidad="pt">36</size>
</font>
```

XML DOM

Existe también un modelo de documento para XML, que facilita su acceso

En Javascript se pueden instanciar objetos de la clase XML DOM con métodos y propiedades que permiten el acceso a las diferentes partes de un documento XML.

```
<Universidad>
  <Santiago>
     <Año>1992</Año>
     <Rector>Antonio Rodríguez</Rector>
  </Santiago>
  <Coruña>
     <Año>1985</Año>
     <Rector>Enrique Pérez</Rector>
  </Coruña>
</Universidad>
```



XML DOM

Existe también un modelo de documento para XML, que facilita su acceso

Los nevegadores tienen clases accesibles mediante Javascript que devuelven objetos de la clase XMLDocument con métodos y propiedades que permiten el acceso a las diferentes partes de un documento XML.

Objeto	Acción
DOMParser	Clase del navegador que instancia objetos un analizador (parser) que convierten un documento de texto en un documento DOM
.parseFromString(texto, tipoMime)	Método de la clase DOMParser que devuelve objetos de la clase XMLDocument, si el texto de entrada es un XML
XMLHttpRequest	Clase del navegador que propiedad responseXML devuelve un objeto XMLDocument

XML DOM

Existe también un modelo de documento para XML, que facilita su acceso

Los objetos de la clase XMLDocument heredan métodos y propiedades de otras clases DOM. Entre los métodos y propiedades más interesantes a este nivel, estarían:

Método	Acción
.getElementsByTagName(nombre)	Devuelve un objeto “NodeList” con todas las etiquetas “nombre”
.appendChild(nodo)	Inserta un nodo hijo al “nodo” actual
.removeChild(nodo)	Inserta un nodo hijo desde el “nodo” actual

Propiedades	Valor
nodeName	Nombre del nodo
nodeValue	Valor del nodo
nodeType	Tipo de nodo (según tabla)
parentNode	Padre del nodo actual
childNodes	Nodos hijo del nodo actual
attributes	Atributos del nodo actual como objeto “node map”

XML DOM

Existe también un modelo de documento para XML, que facilita su acceso

En Javascript existen objeto de la clase XML DOM con métodos y propiedades que permiten el acceso a las diferentes partes de un documento XML.

```
<Universidad>
 ▼<Santiago>
    <Año>1992</Año>
    <Rector>Antonio Rodríguez</Rector>
  </Santiago>
 ▼<Coruña>
    <Año>1985</Año>
    <Rector>Enrique Pérez</Rector>
  </Coruña>
</Universidad>
```

```
.getElementsByTagName("Universidad")[0].childNodes[1].childNodes[0].childNodes[0].nodeValue;
```

↘

1985

XML DOM

Existe también un modelo de documento para XML, que facilita su acceso

En Javascript existen objeto de la clase XML DOM con métodos y propiedades que permiten el acceso a las diferentes partes de un documento XML.

```
<Universidad>
 ▼<Santiago>
    <Año>1992</Año>
    <Rector>Antonio Rodríguez</Rector>
  </Santiago>
 ▼<Coruña>
    <Año>1985</Año>
    <Rector>Enrique Pérez</Rector>
  </Coruña>
</Universidad>
```

```
<h2>Información Universidades</h2>
<p id="uno"></p>
<p id="dos"></p>
```

Información Universidades

Las universidades analizadas son: Santiago, Coruña,

El Rector de la Universidad de Santiago es Antonio Rodríguez

```
let txt_0= xmlDoc.getElementsByTagName("Universidad")[0].childNodes[1].childNodes[1].nodeName;
let txt_1= xmlDoc.getElementsByTagName("Universidad")[0].nodeName;
let txt_2= xmlDoc.getElementsByTagName("Universidad")[0].childNodes[0].nodeName;
let txt_3= xmlDoc.getElementsByTagName("Universidad")[0].childNodes[0].childNodes[1].childNodes[0].nodeValue;
let txt= miF(xmlDoc);

document.getElementById("uno").innerHTML= "Las universidades analizadas son: " + txt;
document.getElementById("dos").innerHTML= "El " + txt_0 + " de la " + txt_1 + " de " + txt_2 + " es  " + txt_3;
```

```
function miF(xml) {
    let txt = "";
    let x = xmlDoc.documentElement.childNodes;
    for (i = 0; i < x.length; i++) {
        if (x[i].nodeType == 1) {
            txt += x[i].nodeName + ", ";
        }
    }
    return(txt);
}
```

JSON

Su nombre deriva de *JavaScript Object Notation*.

Formato para almacenamiento/intercambio de datos, creado inicialmente (1999), en el ámbito de *JavaScript*, pero extendido a múltiples lenguajes en la actualidad.

Es sencillo de manejar por máquinas y humanos.

A efectos de *JavaScript* es un objeto, convertible en cadena de texto para intercambio entre servidores.

JSON

Formato básico

Está construido a partir de dos estructuras básicas:

- Una colección de pares nombre/valor → Dependiendo del lenguaje se implementa como: objeto, registro, estructura, diccionario, tabla hash,…
- Una lista ordenada de valores → En la mayor parte de los lenguajes se implementa como array, vector o lista.

JSON

Formato básico

En JavaScript las dos estructuras son:

- Objetos

{cadena:valor, cadena:valor, …….}

- Arrays

[valor, valor, …..]

```
{
    "Universidad":[
    {"Nombre": "USC",
     "Año": "1992",
     "Rector": "Antonio Rodríguez"},
    {"Nombre": "UDC",
     "Año": "1985",
     "Rector": "Enrique Pérez"}
]}
```

Siendo los valores cadenas, números, objetos, arrays, booleanos o null.

```
miObjeto= {asignatura: "DAW", centro: "ETSE", curso: 4}

miArray= ["Antonio", "José", "Pedro"]
```

JSON

JavaScript cuenta con la clase JSON y métodos para:

- Transformar objetos JavaScript en cadenas JSON

```
var miCadenaJSON= JSON.stringify(miObj)
```

- Transformar cadenas JSON en objetos JavaScript

```
var miObjeto= JSON.parse(miCadenaJSON)
```

JSON

Soporte nativo para JSON en javaScript

```
{
    "Universidad":[
    {"Nombre": "USC",
     "Año": "1992",
     "Rector": "Antonio Rodríguez"},
    {"Nombre": "UDC",
     "Año": "1985",
     "Rector": "Enrique Pérez"}
]}
```

```
<h2>Información Universidades</h2>
<p id="uno"></p>
<p id="dos"></p>
```

Información Universidades

El Rector de la USC es Antonio Rodríguez

El Rector de la UDC es Enrique Pérez

```
jsonObj= JSON.parse(jsonText);

txt_00= jsonObj.Universidad[0].Nombre;
txt_01= jsonObj.Universidad[0].Rector;
txt_10= jsonObj.Universidad[1].Nombre;
txt_11= jsonObj.Universidad[1].Rector;

document.getElementById("uno").innerHTML= "El Rector de la " + txt_00 + " es  " + txt_01;
document.getElementById("dos").innerHTML= "El Rector de la " + txt_10 + " es  " + txt_11;
```

XML vs JSON

XML

```
<alumnos>
    <alumno>
        <nombre>Pedro</nombre>
        <apellido>Gómez<apellido>
    </alumno>
    <alumno>
        <nombre>Pablo</nombre>
        <apellido>Pérez<apellido>
    </alumno>
        …
</alumnos>
```

JSON

```
{"alumnos" : [
   {"nombre" : "Pedro", "apellido" : "Gómez"},
   {"nombre" : "Pablo", "apellido" : "Pérez"},
    …
]}
```

XML vs JSON

Similitudes

Ambos son:

- Autodescriptivos
- Jerárquicos
- Analizables *(Parseables)*
- Tratables por el método XMLHttpRequest de JavaScript

Diferencias

- XML tiene que ser analizado por un analizador (*parser)* XML. JSON puede utilizar funciones Javascript estándar
- JSON no usa etiquetas
- JSON es más rápido, tanto en lectura como en escritura
- JSON puede utilizar arrays

Desarrollo de Aplicaciones Web

Formato Datos: XML, JSON.

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Pablo García Tahoces